Factoring Polynomials: Classical Algorithms, Representations, and Lean/Mathlib

ScottLean4 — tutoring notes

August 4, 2026

Abstract

Two linked questions. *First:* what are the classical algorithms that *factor* a polynomial, organized by coefficient domain, with their costs. *Second:* which of them live in Lean's `Mathlib` — and the answer is essentially *none*, because `Mathlib` carries the *theory* of factorization, not its *algorithms*. The reason is visible in the data structure Lean chose for a polynomial: a finitely-supported coefficient map, optimized for proof rather than computation.

1 Classical factoring algorithms

Factoring splits by coefficient domain. Univariate over a finite field is the best-developed pipeline; everything else reduces toward it.

1.1 Univariate over a finite field $\mathbb{F}_q$

1. **Square-free factorization** (Yun, 1976). Strip repeated factors using $\gcd(f, f')$ (with care in characteristic p, where the derivative can vanish and the Frobenius map intervenes).
2. **Distinct-degree factorization** (DDF). Separate the square-free part into products of irreducibles of equal degree, using $\gcd\big(f,\ x^{q^d} - x\big)$ to peel off the degree-d factors.
3. **Equal-degree factorization** (EDF). Split a product of equal-degree irreducibles.
   - **Cantor–Zassenhaus** (1981): randomized, computing $h^{(q^d-1)/2} \bmod f$ for random h; fast for large q.
   - **Berlekamp** (1967): deterministic, via linear algebra — the *Berlekamp subalgebra* $\ker(\mathrm{Frob} - I)$; good for small q.

1.2 Univariate over $\mathbb{Q}$ and $\mathbb{Z}$

Reduce to $\mathbb{Z}$ by Gauss's lemma (take the primitive part) and make square-free. Then:

1. Factor **modulo a good prime** p (one that keeps the polynomial square-free).
2. **Hensel lifting** (Zassenhaus, 1969): lift the factorization $\bmod p$ to $\bmod p^k$, then *recombine* the lifted factors into true integer factors. Recombination is worst-case exponential.

3. **Lenstra–Lenstra–Lovász (LLL, 1982)**: lattice basis reduction yields the first *polynomial-time* factorization over $\mathbb{Q}$. **van Hoeij** (2002) made recombination practical via knapsack lattices. Consequently, factoring over $\mathbb{Q}$ is in **P**.

1.3 Multivariate

Reduce to the univariate case: **Kronecker substitution** plus **multivariate Hensel lifting** (Wang; Zippel sparse interpolation; Gao). The ancestor is **Kronecker's method** (1882) — the first algorithm, finite but exponential.

1.4 Over $\mathbb{R}$ and $\mathbb{C}$

Exact factorization is **root-finding**: Aberth, Jenkins–Traub, or companion-matrix eigenvalues over $\mathbb{C}$; over $\mathbb{R}$ one lands on linear and irreducible-quadratic factors.

1.5 Cost anchor

Domain	Method	Cost (soft-O)
$\mathbb{F}_q$	DDF + EDF (Cantor–Zassenhaus)	$\tilde{O}(n^2 \log q)$, less with fast arithmetic
$\mathbb{F}_q$	Berlekamp	$O(n^3)$ linear algebra + splitting
$\mathbb{Q}$	LLL / van Hoeij	polynomial time
$\mathbb{Q}$	Zassenhaus (recombination)	exponential worst case

2 Is it in Mathlib? No — theory, not algorithms

A grep of `Mathlib` (v4.32.2) settles it. The classical factoring *algorithms* are absent:

- `Berlekamp` occurs *once*, as a name in `Analysis/Polynomial/MahlerMeasure.lean` — not the algorithm.
- The `Zassenhaus` hits are `SchurZassenhaus` (a group-theory lemma), *not* Cantor–Zassenhaus.
- There is *no* `CantorZassenhaus`, *no* LLL, *no* distinct-degree factorization.

What `Mathlib` *does* carry is the **theory** of factorization:

- `UniqueFactorizationMonoid` — existence and uniqueness of factorization in a UFD (a ∼92-file development);
- `Polynomial.roots` — the multiset of roots, but **`noncomputable`**;
- splitting fields and `Polynomial.factor` — **`noncomputable`**, choosing *an* irreducible factor in order to *construct* a splitting field, not to compute one;
- square-free theory, and irreducibility criteria: Eisenstein (`IsEisensteinAt`), cyclotomic irreducibility, the rational-root theorem;
- **Hensel's lemma as a theorem** (for complete local rings / p-adics), not as a lifting-and-recombination *procedure*.

So computing a factorization is a computer-algebra-system task (FLINT, NTL, Singular, Sage, Magma); Lean's `polyrith` tactic literally shells out to Sage. No *verified* factoring algorithm has been ported into `Mathlib`.

3 Representations of polynomials

Representation	Form	Good at
Dense coefficient	array $[a_0, \ldots, a_n]$	dense polynomials; FFT
Sparse coefficient	list of (exp, coeff)	high degree, few terms
Point–value	$\{(x_i, f(x_i))\}$, $n{+}1$ points	$O(n)$ multiply; FFT $\leftrightarrow$ coeffs in $O(n \log n)$
Product / root	$c \prod_i (x - r_i)$	factored; trivial multiply, hard add
Basis	coeffs in Newton / Bernstein / Chebyshev / Lagrange basis	numerics, CAGD, stability
Multivariate	recursive (nested) vs. distributed (monomials + order)	—

Point–value and the Lagrange basis coincide; the FFT is exactly the fast change of basis between the dense-coefficient and point–value representations, which is why fast multiplication runs in $O(n \log n)$.

4 Lean/Mathlib's representation

Lean uses the abstract **sparse-coefficient representation via `Finsupp`**. A univariate polynomial *is* a finitely-supported map from exponents $\mathbb{N}$ to coefficients R, written $\mathbb{N} \to_0 R$, wrapped in a one-field structure:

```
structure Polynomial (R : Type*) [Semiring R] where ofFinsupp ::
  toFinsupp : AddMonoidAlgebra R Nat     -- AddMonoidAlgebra R Nat  =  (Nat ->0 R)
```

that is, `AddMonoidAlgebra` R $\mathbb{N}$ $=$ $(\mathbb{N} \to_0 R)$. The wrapper (`ofFinsupp` / `toFinsupp`) is deliberate: it blocks bad definitional unfolding and forces use of the API (`coeff`, `degree`, `eval`, `C`, `X`). Multivariate is the same pattern, distributed:

```
abbrev MvPolynomial (sigma R) [CommSemiring R] :=
  AddMonoidAlgebra R (sigma ->0 Nat)     -- = ((sigma ->0 Nat) ->0 R)
```

a `Finsupp` from *multi-exponents* $\sigma \to_0 \mathbb{N}$ to coefficients — the basis-free distributed representation over monomials.

Why this representation. It is chosen for *reasoning*, not *computing*. `Finsupp` gives clean algebra over any semiring, an isomorphism `Polynomial` $R \simeq \bigoplus_{\mathbb{N}} R$, and uniform definitions of `degree`, `coeff`, and `eval`; the module, algebra, and evaluation-homomorphism structure is transparent. The price is that it is largely **`noncomputable`** in practice: `roots` is `noncomputable`; equality needs `DecidableEq R` and even then is not an efficient data structure.

5 Synthesis

The two questions are one fact seen from two sides. `Mathlib` picked the representation that is best for *proofs* — an abstract finitely-supported coefficient map — and it correspondingly carries

the factorization *theory* (unique factorization, roots, splitting fields, irreducibility criteria, Hensel's lemma) but *none* of the computer-algebra *machinery* (dense arrays, sparse term lists, the FFT, Berlekamp, Cantor–Zassenhaus, LLL) that makes factoring fast. Lean's polynomial is a mathematical object to reason about, not a data structure to compute with — which is exactly why the classical factoring algorithms are not, and were never meant to be, in `Mathlib`.